

Experimental Analysis and Assessment of a Real-Life Cloud Mobile Big OLAP System Enhanced with Compression and Approximation Paradigms

Alfredo Cuzzocrea*
iDEA Lab
University of Calabria
Rende, Italy
& Dept. of Computer Science
University of Paris City
Paris, France
alfredo.cuzzocrea@unical.it

Mojtaba Hajian
iDEA Lab
University of Calabria
Rende, Italy
mhajian.idealab.unical@gmail.com

Abstract—The increasing complexity and volume of data in Cloud-based environments have posed significant challenges for *Online Analytical Processing (OLAP)*, especially on resource-constrained mobile devices. This paper introduces the *Indexed Quad-Tree Summary (IQTS)* algorithm, a novel technique for compressing and approximating *multidimensional OLAP data cubes*. IQTS efficiently handles large data cubes by flattening dimensions, employing *Quad-Tree partitioning*, and leveraging efficient *Indexing methods*. These processes significantly reduce storage requirements while ensuring high accuracy in query responses. Our experimental evaluation, utilizing real-world medical datasets, demonstrates that IQTS outperforms existing data compression techniques such as *MinSkew*, *STHoles*, and *GenHist*, delivering superior results in terms of space efficiency and query accuracy. The paper demonstrates the algorithm’s potential for mobile Cloud environments, showing its scalability and adaptability for big data applications.

Keywords—Big Data, Big OLAP Cubes, Mobile Cloud Systems, Compression and Approximation Paradigms

I. INTRODUCTION

The rapid growth of data in various sectors has led to significant challenges in data storage, processing, and analysis, particularly in Cloud environments. One critical domain that requires efficient data handling is *Online Analytical Processing (OLAP)*, widely used for multidimensional data analysis. OLAP systems are integral to applications such as decision support, data mining, and business intelligence, where large datasets are queried to gain insights. However, as the volume and complexity of data continue to increase, traditional OLAP systems become computationally expensive and inefficient, particularly when used on mobile devices that have limited computational resources and storage (e.g., [1-3]).

Cloud computing has become a key solution in overcoming the limitations of *mobile OLAP (mOLAP)* by offering scalable storage and computational resources, allowing businesses to centralize and access vast amounts of data efficiently (e.g., [4,5]). However, the increasing number of OLAP cubes in Cloud environments presents challenges in selecting the most relevant cubes for analysis. To address this, researchers have developed recommendation systems that help users identify and merge relevant cubes, simplifying data exploration and improving analytical efficiency [6]. In this context, personalization and creating customized OLAP cubes have emerged as a vital feature in OLAP systems, particularly

in mobile contexts ([7,8]). Models like *CobWeb* enable flexible, user-adaptive OLAP analyses, dynamically generating schemas based on user needs. These advancements enhance decision-making by minimizing the time spent searching for relevant data [9].

Despite these innovations, mOLAP still faces challenges, particularly in terms of maintaining performance under resource constraints. The *m-Dwarf* structure proposed by [10], for example, addresses this by compressing data cubes to fit the limited storage capacity of mobile devices, but at the cost of increased client-side processing. This trade-off between storage efficiency and processing power highlights the ongoing need for optimized solutions that balance these competing demands in mobile environments.

Furthermore, mOLAP systems must account for varying network conditions, which can impact data transmission and query performance. In this context, hybrid broadcast architectures, which combine *push-based* and *pull-based* data dissemination techniques, offer a potential solution for improving data accessibility under fluctuating network conditions ([10-12]). Besides, *Optimal Query Mapping* introduces a method of mapping queries to aggregation lattices, exploiting sub-cube dependencies in multidimensional databases. This technique reduces the number of handled data items, optimizing both scheduling and dissemination and ensuring efficient transmission under bandwidth constraints [13]. These architectures allow mobile devices to retrieve data more efficiently by broadcasting commonly requested data, reducing the frequency of client-server interactions.

This paper addresses the challenges associated with executing complex OLAP queries on mobile devices by introducing a novel data compression and query approximation technique known as the *Indexed Quad-Tree Summary (IQTS)*. The IQTS algorithm is designed to handle large, multidimensional OLAP data cubes by reducing their dimensionality, compressing the data, and enabling efficient query processing through advanced compression techniques. By compressing the data into a two-dimensional structure and employing *Quad-Tree partitioning*, IQTS facilitates fast and accurate query responses while significantly reducing storage requirements. The paper extends and integrates the paper [14], where we introduced the algorithmic technique for creating the IQTS compressed representation of OLAP data cubes, and the advanced query model for querying compressed OLAP cubes. Here, we first provide an overview of IQTS, and then focus on its comprehensive experimental campaign.

* This research has been made in the context of the Excellence Chair in Big Data Management and Analytics at University of Paris City, Paris, France

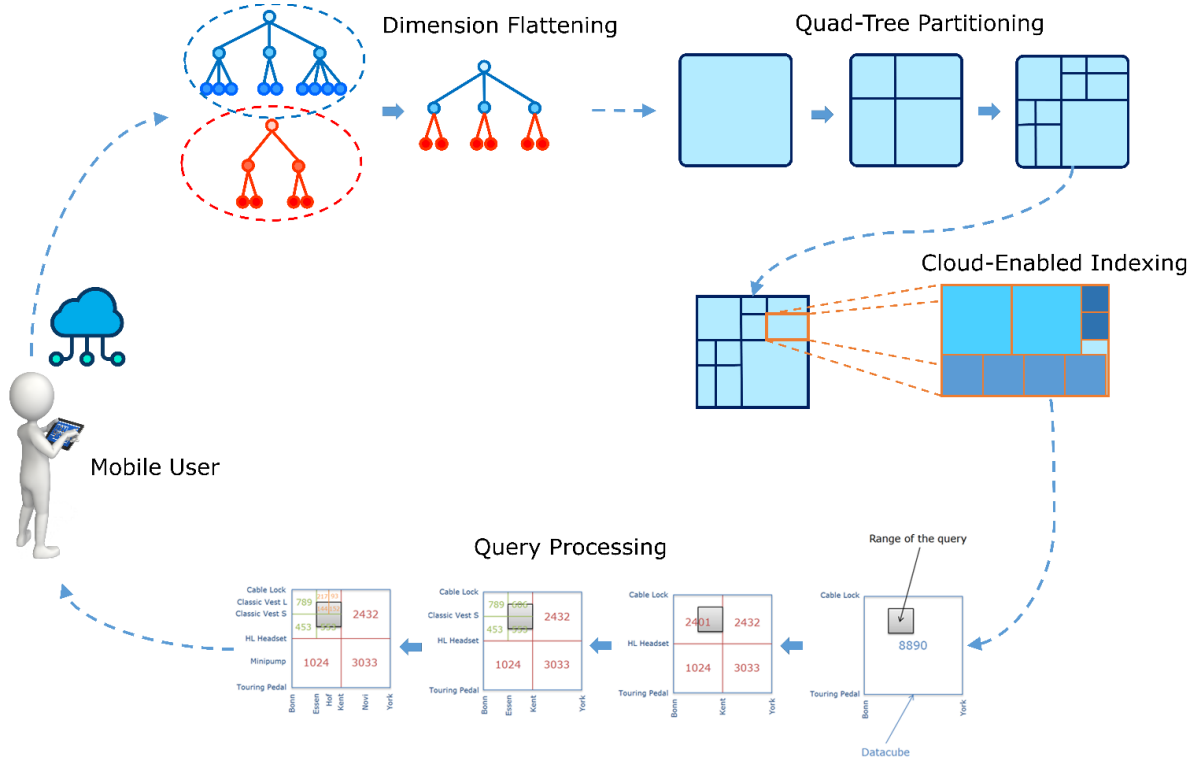


Fig. 1. Architecture of IQTS

The remainder of the paper is organized as follows: Section II provides a detailed overview of the IQTS algorithm, describing its key components, including dimension flattening, quad-tree partitioning, and Indexing methods. Section III focuses the attention on significant amenities of the whole IQTS proposal. Section IV provides the description and the organization of our experimental campaign. Section V describes each-dataset characteristics and details. Section VI turns the attention on two-dimensional OLAP views obtained from the considered datasets. Section VII presents the experimental settings in details. Section VIII discusses the experimental results, comparing the performance of IQTS with existing techniques. Finally, Section IX concludes the paper and discusses potential future work.

II. IQTS IN A NUTSHELL

Typical OLAP systems, nowadays, have high dimensions which makes them too much complicated and computationally expensive for analytics, which prevents them from being used in mobile devices with their low computational resources. One innovative idea for handling this issue is utilizing compression techniques applied to OLAP data cubes to enable m -users (mobile users) to explore the complex OLAP data cube based on their needs.

The *Indexed Quad-Tree Summary* algorithm represents a significant advancement in the domain of data compression, particularly concerning multidimensional OLAP data cubes [14]. This technique is designed to address the challenges posed by the increasing complexity and size of data, especially within the context of mobile Cloud environments where computational resources and storage are often limited. The following Section provides a comprehensive overview of the IQTS algorithm, delving into its core components, including dimension flattening, quad-tree partitioning, and the Indexing

methods exploitation, which collectively contribute to its efficacy in data compression and query approximation.

IQTS operates through a series of methodical steps that transform a multidimensional OLAP data cube into a compressed, two-dimensional representation that can be efficiently managed by mobile devices. The structure of IQTS consists of two main components: (i) *OLAP Dimension Flattening*; (ii) *OLAP Data Cube Partitioning*; (iii) *Indexing Methods*. We will now walk through each of these as follows. A comprehensive illustration of the architecture of IQTS is shown in Fig. 1. Now we walk through each of the components.

A. OLAP Dimension Flattening

The first step in the IQTS compression process is *Dimension Flattening*, which is crucial for managing the high dimensionality of OLAP data cubes. Data cubes in OLAP systems often contain multiple dimensions, each representing different attributes or variables in the dataset. The challenge with high-dimensional data is the curse of dimensionality, which refers to the exponential increase in data volume as the number of dimensions increases. This can lead to sparsity, making it difficult to perform efficient data analysis and compression.

Dimension flattening addresses this issue by reducing the number of dimensions in the data cube. The process involves merging two dimensions into a single dimension, thereby simplifying the structure of the data cube. For example, consider a data cube with dimensions d_k and d_{k+1} , each with its own hierarchical structure denoted as $H(d_k)$ and $H(d_{k+1})$, respectively. Given these two selected dimensions, the IQTS algorithm selects a *sub-tree* from the hierarchy of $H(d_{k+1})$ and merges it into $H(d_k)$, resulting in a new, merged dimension $H^*(d_k)$. This process is repeated for other

dimensions, effectively flattening the multidimensional space into a two-dimensional representation. After the dimension flattening process is complete, it is essential to re-aggregate the data according to the new, flattened hierarchy. This step is critical to ensure that the data remains consistent and accurate within the new structure, enabling efficient OLAP queries and analysis. For the sake of simplicity, here we reduce the number of dimensions into two dimensions, thus, eventually, we obtain a new two-dimensional data cube from the original multidimensional OLAP data cube.

B. Quad-Tree Partitioning

The *OLAP Data Cube Partitioning* technique is responsible for managing and compressing large data cubes in a way that maintains the integrity of the data while enabling efficient querying. The exploitation of quad-trees and iterative partitioning allows for a balanced and optimized structure that can be effectively compressed using advanced Indexing techniques.

This part describes the main compression technique applied to a two-dimensional OLAP data cube C after dimension flattening. The objective is to compress the data cube by developing a *quad-tree partition*, $P(C)$, where each *bucket* (i.e., a partition of the cube) contains the sum of the items within it [15]. These buckets are generally *rectangular*, which provides an arbitrary shape selection for the buckets' dimensions, and the partitioning process is driven by the input (merged) dimensions.

The *quad-tree partitioning* algorithm begins by considering the entire two-dimensional representation of the data cube as a single bucket. This bucket is then subjected to an iterative partitioning process, where it is repeatedly divided into smaller sub-buckets. The goal of this partitioning process is to create a hierarchical structure where each node in the quad-tree represents a bucket, and the leaves of the tree represent the smallest sub-buckets.

In more detail, the *splitting step* seeks the optimal position, $O_{H^*(d_k)}$, for the given merged dimension hierarchy $H^*(d_k)$. This is achieved by first identifying a halfway point, P_k , within a specified interval $[l_k:r_k]$ of the given merged dimension. The position is computed as follows:

$$P_k = \left\lfloor \frac{r_k - l_k + 1}{2} \right\rfloor \quad (1)$$

The algorithm then selects a node at a specific level L_k of the merged hierarchy $H^*(d_k)$, and designate it by β_k , such that it is an ancestor node of α_k^* . Here, α_k^* is either the *Leftmost Descendant Leaf* (LDL) α_k^L or the *Rightmost Descendant Leaf* (RDL) α_k^R . The current splitting position, α_k^* , is determined based on whether it falls inside or outside a specified interval θ_k , as follows:

$$\theta_k = \left[\begin{array}{l} H_k - \left\lfloor \frac{1}{3}(r_k - l_k + 1) \right\rfloor \\ H_k + \left\lfloor \frac{1}{3}(r_k - l_k + 1) \right\rfloor \end{array} \right] \quad (2)$$

where H_k is the current *halfway point*.

This procedure starts from β_k , as the root level of the merged hierarchy and continues to deeper levels of the hierarchy iteratively until finds a α_k^* inside θ_k , which can be considered as $O_{H^*(d_k)}$. This method targets to identify a

splitting position close to the halfway point of the interval $[l_k:r_k]$, to ensure that the data cube is divided in a way that maintains the integrity of the data while optimizing the structure for storage and query performance. By carefully choosing where to split the data, the algorithm minimizes the loss of information and ensures that the resulting partitions are both manageable in size and efficient to work with.

C. Indexing Methods

This step of our compression algorithm particularly focuses on the output after the data cube has been partitioned. This process allows for optimizing both storage and query performance. After the OLAP data cube has been partitioned into smaller *leaf buckets*, the next step is to further compress these leaf buckets. After the partitioning process is finished, the algorithm deals with the challenge of efficiently storing these partitions using *Indexing* methods. It assumes that OLAP data cubes consist of non-negative integers and that each bucket contains a sum of the measure values, stored using 64 bits. If a bucket contains zero values, these bits are left null to save space. Moreover, since the fourth sub-bucket's value can be computed by subtracting the sum of the three sub-buckets from the parent bucket's value, the algorithm doesn't keep the value of fourth sub-bucket for further efficiency in memory consumption.

These Index data structures help approximate the aggregate data within each sub-bucket, ensuring that the compressed data remains accurate in the query step. The final structure, stored in memory, includes both the sum values (*Sum Array*) and the structural details of the partitioning process (*Structure Array*).

D. Query Processing in Compressed OLAP Views

Here, we explain how range queries are processed on compressed OLAP views, particularly when dealing with the approximate nature of the data stored in buckets within a quad-tree structure. The process involves using linear interpolation and various compression techniques to answer queries efficiently, even when the data distribution inside each bucket may not be uniform.

When a range query is executed on a compressed OLAP view, the query algorithm navigates the quad-tree to identify two types of buckets: (i) *Completely Involved Buckets*; these are buckets that are fully contained within the query range. The sum of the values within these buckets is stored directly, so the query results for these buckets are exact, with no approximation required. (ii) *Partially Involved Buckets*; these are buckets that partially overlap with the query range and are not further split into smaller partitions. Since these buckets are only partially included in the query, their contribution to the query result is approximate.

For the partially involved buckets, the algorithm uses linear interpolation based on the assumption that data is uniformly distributed within the bucket, a concept known as the *Continuous Value Assumption* (CVA) [16]. The approximate answer to the query (e.g., [17]), denoted as $\tilde{Q}(b)$, is computed using the following formula:

$$\tilde{Q}(b) = \frac{|Q \cap b|}{|b|} \cdot S(b) \quad (3)$$

where $S(b)$ is the sum of the values in bucket b , $|Q \cap b|$ is the area of the intersection between the query range Q and the bucket b , and $|b|$ is the area of bucket b .

The implementation of this method involves associating an *Index* with each leaf bucket of the quad-tree to describe the internal distribution of data approximately. The *Index* provides a compact representation of the sums within sub-buckets and sub-sub-buckets, ensuring that the original data's accuracy is retained as much as possible. The *Index*-based approximation reduces the error in query estimates, particularly when the data distribution is not uniform.

III. FURTHER REMARKS

To wrap up, the IQTS algorithm offers several distinct advantages that make it a valuable tool for data compression and query processing in OLAP systems. These benefits are particularly discussed as follows:

- *Efficient Storage*: By compressing the data cube into a hierarchical structure of quad-trees and Index data structures, IQTS significantly reduces the storage requirements. This is especially beneficial in mobile environments where storage capacity is limited. The algorithm ensures that only the most important information is retained, minimizing the amount of data that needs to be stored.
- *Accurate Approximate Query Answering*: Despite the significant compression of the data, IQTS is capable of delivering accurate query results. The algorithm is capable of approximating query results with a high degree of accuracy, even when the data is not uniformly distributed. Additionally, IQTS provides *Enhanced Query Performance* which enables efficient query processing by reducing the number of data partitions that need to be accessed for each query.
- *Scalability*: IQTS is designed to handle large-scale data cubes with high dimensionality. The algorithm's ability to flatten dimensions and partition data into manageable partitions makes it well-suited for processing big data in mobile Cloud environments.
- *Flexibility*: The Indexing methods employed in the algorithm allow IQTS to adapt to different data distributions and query requirements. By selectively partitioning and compressing the data, the algorithm can optimize its performance based on the specific characteristics of the data cube.

IV. EXPERIMENTAL ASSESSMENT AND ANALYSIS:

MOTIVATIONS AND ORGANIZATION

In this Section, we describe the experiments conducted to evaluate the efficiency of the IQTS algorithm. We begin by detailing the datasets used, which are relevant to the field of medicine. For each dataset, we provide a brief overview of the associated disease and explain the modifications made to adapt the datasets to our case study. The attributes of each dataset are then presented in tabular form, along with their respective descriptions. Additionally, we describe the process by which we obtained the two-dimensional visualizations fundamental to our analysis, supported by appropriate explanatory figures illustrating the data distribution. We then focus on the experimental metrics employed, providing the formulas utilized in our analysis. Finally, we present the results obtained from the experiments. We compare the performance of IQTS with other leading compression techniques, specifically the *MinSkew* [18], *STHoles* [19], and *GenHist* [20] algorithms. The comparison is based on the

results of experiments using two different value scales, with the criteria for comparison clearly defined.

In the last Section, we demonstrate how IQTS outperforms the other techniques across all metrics, confirming the superior compression capability of the process we have developed.

V. DATASETS DESCRIPTION AND ANALYSIS

In this Section, we provide a detailed description of the datasets exploited by our experimental campaign, which plays a leading role in our research.

For the analysis of the experimental results, we utilize two real-life datasets from the field of medicine, sourced from the *University of California-Irvine Department of Information and Computer Sciences repository*. The selected datasets are as follows:

- *Echocardiogram* [21];
- *Cardiotocography* [22].

However, the cardinality of *Echocardiogram* dataset is lower than desired for reliable analysis, while the other dataset contains more data. To address this, we reduce the number of tuples in the *Cardiotocography* dataset to 2,000 and increase the cardinality of the smaller dataset to achieve more satisfactory levels. This expansion process is performed using customized tools developed specifically for each case. For each dataset, we extract the attribute values, and for numerical attributes, we compute the maximum and minimum limits. Based on the range of the numerical fields, we determine the interval between values that will yield the desired number of tuples using the formula: $(max - min / number)$. New tuples are then generated with unique values for the expanded fields, while values for other attributes are replicated sequentially to complete the dataset. A final shuffle operation is performed to randomize the order of the tuples. The modified cardinalities of the datasets are as follows:

- *Echocardiogram* changes from cardinality 133 to 1500;
- *Cardiotocography* changes from cardinality 2127 to 2000.

We now look at these datasets in more detail. For example, the *Echocardiogram* dataset pertains to a diagnostic test that provides visualizations of the patient's heart, enabling the physician to assess whether the heart's structure and function are normal or if any issues are present. Tab. I outlines the attributes stored in the corresponding *Echocardiogram* dataset.

Cardiotocography is a widely used tool in *obstetrics* for assessing fetal well-being in the perinatal setting and monitoring the presence, frequency, and intensity of uterine contractions during childbirth. This examination serves two primary objectives: (i) to evaluate fetal well-being through the analysis of the fetal heartbeat; (ii) to detect the presence or absence of uterine contractions along with their frequency. Tab. II provides an overview of the fields stored in the corresponding *Cardiotocography* dataset.

It should be noted, here, that the datasets we exploit in our experimental analysis expose all the complexities that are typical of real-life settings, especially within the emerging context of big data processing and analytics.

TABLE I. ATTRIBUTES OF ECHOCARDIOGRAM DATASET

Attribute	Description
Alive_At_1	Binary value: 0 indicates that the patient died within one year of the examination, 1 implies that after a year he was still alive.
Age_At_Heart_Attack	If the patient has had a heart attack, indicate the age he was when the event occurred.
EPSS	Measurement of the contractility of the patient's heart (septal separation from point E), high values indicate cardiac anomalies.
Fractional_Shortening	Measurement of the contractility of the patient's heart (fractional shortening), low values indicate cardiac abnormalities.
Group_At_1	Establishes the type of patient
LVDD	Left Ventricular end-Diastolic Dimension measurement, high values generally indicate diseased hearts.
Name	Patient name.
Survival	Number of months of life attributed to the patient (if still alive), applicable only for some types (used in combination with the Alive-At-1 and Group-At-1 attributes)
Pericardial_Effusion	Binary value that indicates a collection of fluid around the heart (pericardial effusion), 0 if there were no effusions, 1 otherwise
Still_Alive	Binary value: 0 indicates that the patient died at the end of the survival period that was attributed to him (Survival attribute), 1 that he is still alive
Wall_Motion_Index	Index obtained by dividing Wall-Motion-Score by the number of visible segments
Wall_Motion_Score	Measurement of how the segments of the left ventricle are moving (the ventricular profile is usually divided into zones called segments, usually 12-13 are observed with the echocardiogram)

TABLE II. ATTRIBUTES OF CARDIOTOCOGRAPHY DATASET

Attribute	Description
AC	Number of Fetal Accelerations per Second
ALTV	Percentage of time in which the fetus's heartbeat has undergone an abnormal change of prolonged duration
ASTV	Percentage of time in which the fetus's heartbeat has undergone an abnormal change of short duration
Class	Fetal heart rate class
DL	Number of short fetal accelerations per second
DP	Number of sustained fetal accelerations per second
DS	Number of strong fetal accelerations per second
FM	Number of fetal movements per second
LB	Basic fetal heart rate value, measured in beats per minute
Mean	Average value in the fetal heart rate histogram
Median	Median value in the fetal heart rate histogram
Min	Minimum value in the fetal heart rate histogram
Max	Maximum value in the fetal heart rate histogram
MLTV	Average value of the time in which the heartbeat of the fetus has undergone a variation of prolonged duration
Mode	Type of histogram used to monitor fetal heart rate
MSTV	Average value of the time in which the heartbeat of the fetus has undergone a change of short duration
Nmax	Number of peaks present in the fetal heart rate histogram
NSP	Code that identifies the class of the state of the fetus
Nzeros	Number of zeros in the fetal heart rate histogram
Tendency	Trend value in the fetal heart rate histogram
UC	Number of uterine contractions per second
Variance	Value of variance in the fetal heart rate histogram
Width	Amplitude of the fetal heart rate histogram

VI. TWO-DIMENSIONAL OLAP VIEWS DESCRIPTION AND ANALYSIS

A detailed analysis of the two-dimensional OLAP views generated from two distinct medical datasets is presented in this Section. By utilizing different combinations of dimensions and measures for each dataset, we construct two-dimensional views that reveal the underlying data patterns. These views are then used as target multidimensional big datasets for running the IQTS compression algorithm.

This way, we are able to stress the performance of IQTS on multidimensional data structures directly. The latter is a relevant problem, at now (e.g., [23-25]), since nowadays big data systems are more and more connected with *autonomous operational systems* that fulfill data directly to them via multidimensional OLAP analysis procedures (e.g., [26,27]), for further analytical processing and decision-making.

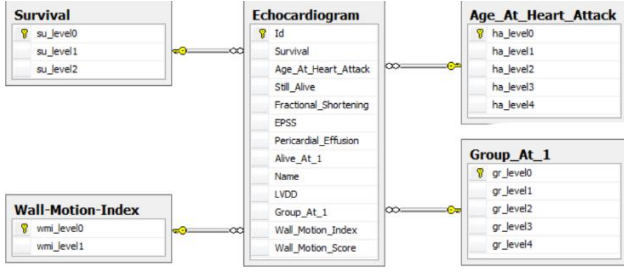


Fig. 2. Two-Dimensional OLAP View on the *Echocardiogram* Dataset

For the *Echocardiogram* dataset (see Fig. 2), we create the two-dimensional view by selecting the *Age_At_Heart_Attack* and *LVDD* attributes as dimensions, with the *Survival* attribute serving as the measure.

On the other hand, for the *Cardiotocography* dataset (see Fig. 3), the two-dimensional view is constructed using the *Mean* and *Width* fields as dimensions, with the *FM* attribute selected as the measure of interest.

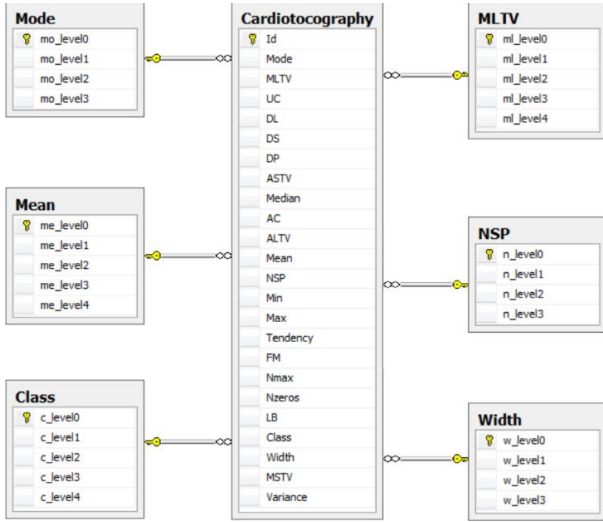


Fig. 3. Two-Dimensional OLAP View on the *Cardiotocography* Dataset

VII. EXPERIMENTAL SETTINGS

In the experiments, we employed *range-sum* queries categorized into two sets based on query size (M), denoted as $QS_{M,R}$ and $S_{M,S}$. The prior includes queries where each interval is randomly generated, ensuring no overlap among them. These queries involve unique rectangular areas, covering the entire compressed data cube. The latter is constructed using a method that creates a set of queries with uniform range sizes and increasing coverage, ensuring non-repetition of areas. The $QS_{M,R}$ examined random query handling, while $QS_{M,S}$ examined scalability with increasing input size.

The accuracy of the techniques was measured using the compressed data size and average relative error ($\bar{E}_{rel}$), computed as the difference between exact and approximate answers, which is considered as error for individual queries. We also measured *space gain* (G), inversely related to *storage space* (B), where higher compression led to reduced accuracy.

As mentioned in Section IV, the experimentation conducted involved a comparative analysis of the IQTS compression algorithm, against the primary alternatives: *MinSkew*, *STHoles*, and *GenHist*. In the following, we report

on the trends in average error relative to query selectivity, the amount of data involved, and the space occupied by the compressed representation. During the variation analysis of each parameter, the other was held constant, with two different scales of values used for this purpose. Finally, in order to achieve a *finer* experimental analysis, we divided our campaign in two variants, based on the width of the intervals of the independent (experimental) parameters, namely *small-scale* and *large-scale*, respectively.

VIII. EXPERIMENTAL RESULTS

Now, we focus the attention on experimental results. When setting the selectivity of the queries on the two-dimensional view of the *Echocardiogram* dataset at 22.5%, the average error observed in the compression techniques under examination exhibits some irregularities. Specifically, when allocating more memory space to the compressed representations, the approximation error does not immediately decrease as expected. Instead, it tends to remain constant or even increase slightly, only starting to decline at the end of the scale. In Fig. 4, it is evident that the IQTS algorithm initially produces an error greater than 15%, which peaks when the allocated memory reaches 35%, before reducing to approximately 15% at 40% allocated space. The other techniques follow a similar trend, generally producing a slightly higher error, with the exception of *STHoles*. *STHoles* demonstrates a parabolic trend with a downward-facing concavity, reaching a maximum error of 45% at its vertex.

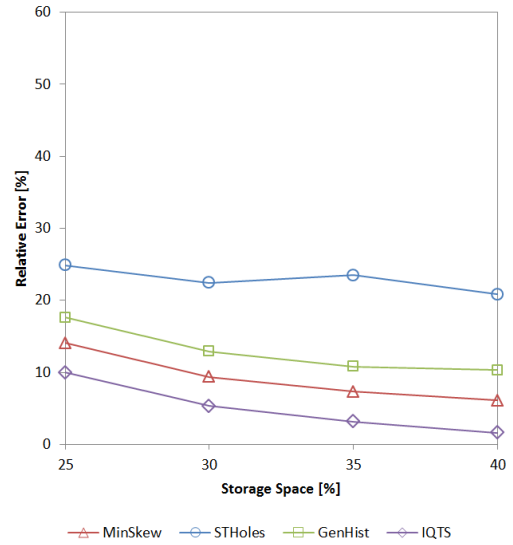


Fig. 4. Experimental Results of *Storage Space* on the *Echocardiogram* Two-Dimensional View (Small-Scale)

When considering queries that involve increasingly large data areas and fixing the memory space for storing the compressed data at 32.5%, the mean error increases, as depicted in Fig. 5. However, IQTS demonstrates strong performance under these conditions, effectively managing the increase in query size while maintaining a relatively low error rate.

Fig. 6 illustrates that for queries involving 22.5% of the data in the two-dimensional view of the *Echocardiogram* dataset when the compressed histogram occupies 45% of the original data's memory, the IQTS compression algorithm achieves an average relative error of approximately 10%. This error tends to decrease as the space allocated to the

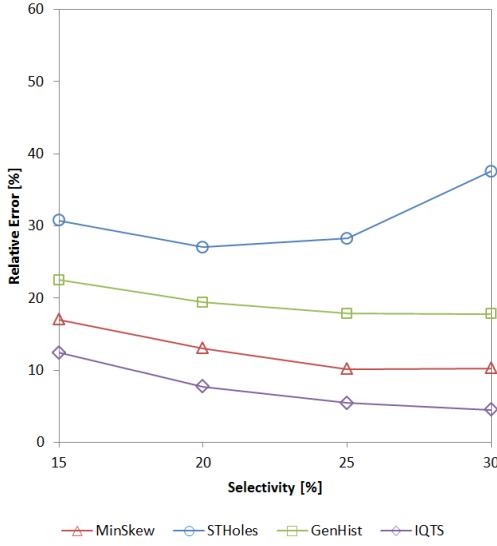


Fig. 5. Experimental Results of *Selectivity* on the *Echocardiogram* Two-Dimensional View (Small-Scale)

compressed representation increases. In contrast, the other techniques exhibit higher errors: *MinSkew* and *GenHist* begin with errors slightly below 15% and 20%, respectively, and, following a trend similar to *IQTS*, stabilize at final values close to 10%. *STHoles*, however, performs the worst, starting with an initial error of 25%. Even as more memory is allocated, the error reduction is minimal, resulting in an approximation error greater than 20% at the upper end of the memory scale.

The plot in Fig. 7, which is generated by setting the memory limit for storage at 32.5% and increasing the selectivity of the queries, highlights a relatively uniform internal data distribution. In this scenario, all compression techniques, except for *STHoles*, maintain an almost linear trend, with *IQTS* consistently achieving the lowest error.

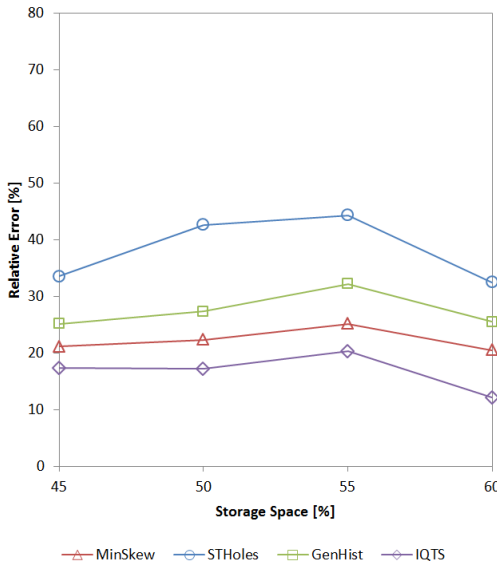


Fig. 6. Experimental Results of *Storage Space* on the *Echocardiogram* Two-Dimensional View (Large-Scale)

Regarding the two-dimensional view of the *Cardiotocography* dataset, all the methods demonstrate higher approximation errors compared to previous cases. However, Fig. 8 shows that the *IQTS* algorithm reveals excellent

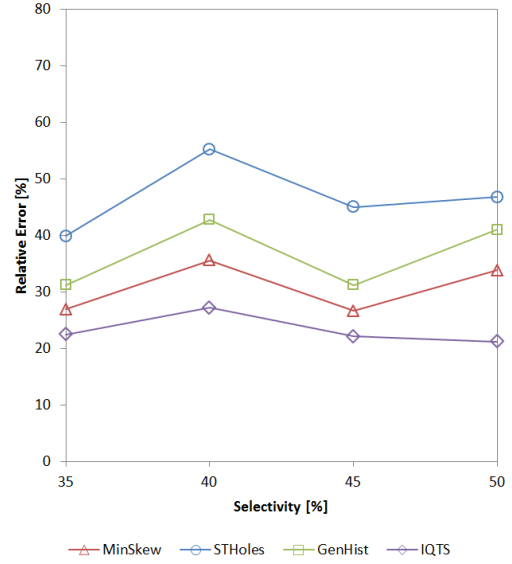


Fig. 7. Experimental Results of *Selectivity* on the *Echocardiogram* Two-Dimensional View (Large-Scale)

accuracy. As the memory space allocated for compressed representations increases from 25% to 30%, the initially high error of 25% falls significantly, approaching 10%. This lower level of approximation remains stable up to 35% of the available space, but then it increases, reaching approximately 18% error at full scale. In this scenario, all the algorithms, including *STHoles*, show relatively stable trends, but they do not match the accuracy of the *IQTS* technique.

When the compressed data occupies 32.5% of the memory space of the two-dimensional view and queries affect 35% of the view, the error remains relatively high. As shown in Fig. 9, across a range of 36% to 50%, all techniques reveal a linear decrease in error as the memory allocation increases to 45%, after which the error rises again, except for *STHoles*, which remain constant at around 50%.

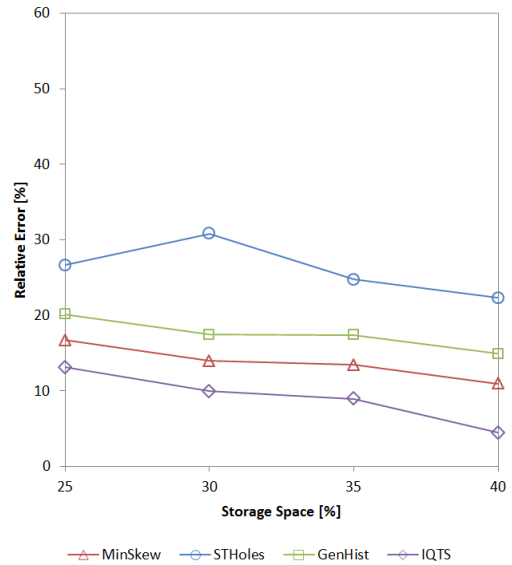


Fig. 8. Experimental Results of *Storage Space* on the *Cardiotocography* Two-Dimensional View (Small-Scale)

The results presented in Fig. 10 demonstrate a very desirable level of approximation across all the techniques examined, with particularly strong performance from *IQTS*,

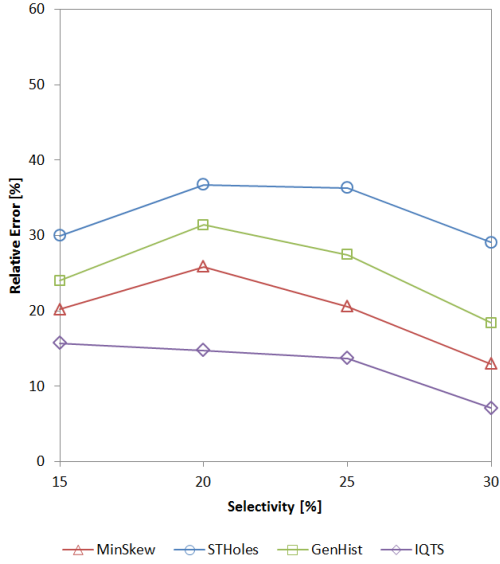


Fig. 9. Experimental Results of *Selectivity* on the *Cardiotocography* Two-Dimensional View (Small-Scale)

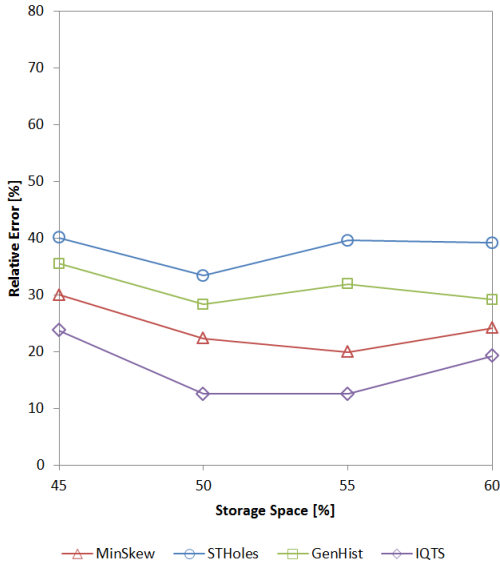


Fig. 10. Experimental Results of *Storage Space* on the *Cardiotocography* Two-Dimensional View (Large-Scale)

MinSkew, and *GenHist*. In contrast, *STHoles* reveals a peak in error when the memory space allocated reaches 45% of that occupied by the two-dimensional view. The errors produced by the other methods range between 12% and 20%, with *IQTS* consistently achieving the lowest values, thereby ensuring the best overall performance.

When analyzing the selectivity plot in Fig. 11, the compression capabilities of the *IQTS* technique become even more evident. For queries involving 35% of the data, the error remains around 15% and stays constant up to 45% of the available memory, after which it falls significantly to just over 5% at full scale. In contrast, *MinSkew*, *GenHist*, and *STHoles* show an arched trend, where the error increases when queries target regions covering between 40% and 45% of the total area, before rapidly decreasing in the final part of the scale, where the space extension reaches 50%.

From the analysis of all the presented experimental results, the benefits due to *IQTS* for big data are clearly evident.

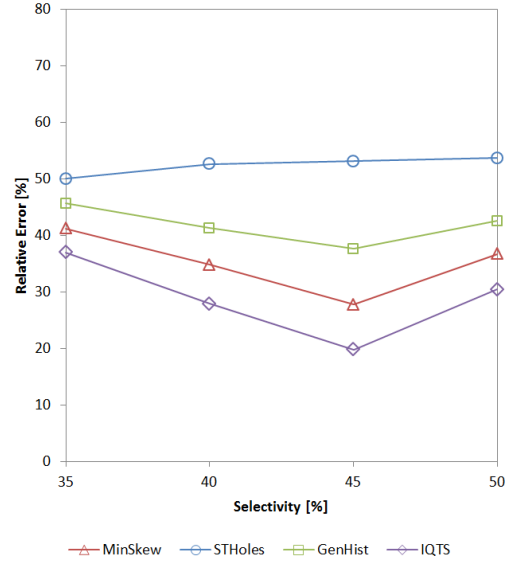


Fig. 11. Experimental Results of *Selectivity* on the *Cardiotocography* Two-Dimensional View (Large-Scale)

IX. CONCLUSIONS AND FUTURE WORK

In this paper, we presented the *Indexed Quad-Tree Summary* algorithm, a novel data compression and query approximation technique specifically designed for large, multidimensional OLAP data cubes. The primary objective of *IQTS* is to address the challenges of executing complex OLAP queries on *resource-constrained mobile devices*. By leveraging dimension flattening, quad-tree partitioning, and Indexing methods, *IQTS* significantly reduces the data storage requirements while maintaining high query accuracy. Our experimental results demonstrate the efficiency of *IQTS* in compressing OLAP data cubes, particularly when compared to existing techniques such as *MinSkew*, *STHoles*, and *GenHist*. *IQTS* consistently outperforms these methods across various metrics, including space efficiency, selectivity, and relative error of the approximate answer of the compressed data cube. The ability to maintain a *low approximation error* while managing large-scale data cubes underscores the effectiveness of the *IQTS* algorithm for Cloud-based mobile OLAP systems.

For future work, we are focusing the attention on extending our algorithm towards innovative aspects of the proper big data processing aspect, such as *semantics analysis* (e.g., [28]), *sensor data integration* (e.g., [29]), *client-based personalization* (e.g., [30]), and *complex data formats* (e.g., [31,32]).

ACKNOWLEDGMENT

This work was funded by the Next Generation EU – Italian NRRP, Mission 4, Component 2, Investment 1.5 (Directorial Decree n. 2021/3277) - project Tech4You n. ECS0000009.

REFERENCES

- [1] A. Cuzzocrea, F. Furfaro, G.M. Mazzeo, D. Saccà, "A Grid Framework for Approximate Aggregate Query Answering on Summarized Sensor Network Readings". In: *OTM Confederated International Workshops*, pp. 144-153, 2004
- [2] G. Marketos, Y. Theodoridis, "Ad-Hoc OLAP on Trajectory Data". In: *11th IEEE International Conference on Mobile Data Management*, pp. 189-198, 2010

- [3] A. Cuzzocrea, R. Moussa, G. Xu, "OLAP*: Effectively and Efficiently Supporting Parallel OLAP over Big Data". In: *3rd International Conference on Model and Data Engineering*, pp. 38-49, 2013
- [4] J. Li, H. Zhou, W. Wang, "Gradual Cube: Customize Profile on Mobile OLAP". In: *6th IEEE International Conference on Data Mining*, pp. 943-947, 2006
- [5] D.R. Nachida, F. Nader, F. Boumahdi, "A New Approach to Supporting Runtime Decision Making in Mobile OLAP". *International Journal of Information and Communication Technology* 13(2), pp. 149-175, 2018
- [6] R. Djioun, M.A. Guessoum, K. Boukhalfa, E. Benkhelifa, "Towards an OLAP Cubes Recommendation Approach in Cloud Computing Environment". In: *8th IEEE International Conference on Social Network Analysis, Management and Security*, pp. 1-6, 2021
- [7] J. Li, H. Zhou, W. Wang, "Gradual Cube: Customize Profile on Mobile OLAP". In: *6th IEEE International Conference on Data Mining*, pp. 943-947, 2006
- [8] R. Nachida, N. Fahima, "A Contextual Personalized Recommender System for Mobile OLAP". In: *IEEE International Conference on Information Technology and e-Services*, pp. 1-7, 2012
- [9] O. Khrouf, K. Khrouf, J. Feki, "CobWeb Multidimensional Model: From Modeling to Querying". In: *4th International Conference on Model and Data Engineering*, pp. 273-280, 2014
- [10] I. Michalarias, A. Omelchenko, "Compressed Aggregations for Mobile OLAP Dissemination". In: *18th IEEE International Workshop on Database and Expert Systems Applications*, pp. 609-614, 2007
- [11] I. Michalarias, A. Omelchenko, H. Lenz, "FCLOS: A Client-Server Architecture for Mobile OLAP". *Data & Knowledge Engineering* 68(2), pp. 192-220, 2009
- [12] M.A. Sharaf, P.K. Chrysanthis, "Facilitating Mobile Decision Making". In: *2nd ACM International Workshop on Mobile Commerce*, pp. 45-53, 2002
- [13] I. Michalarias, H. Lenz, "Optimal Query Mapping in Mobile OLAP". In: *11th East European Conference on Advances in Databases and Information Systems*, pp. 250-266, 2007
- [14] A. Cuzzocrea, M. Hajian, "Effective and Efficient Big OLAP Data Cube Compression in Mobile Cloud Environments: The IQTS Algorithm". In: *IEEE International Conference on Big Data*, pp. 5187-5196, 2023
- [15] F. Buccafurri, F. Furfaro, G.M. Mazzeo, D. Saccà, "A Quad-Tree based Multiresolution Approach for Two-Dimensional Summary Data". *Information Systems* 36(7), pp. 1082-1103, 2011
- [16] G. Colliat, "OLAP, Relational, and Multidimensional Database Systems". *SIGMOD Record* 25(3), pp. 64-69, 1996
- [17] A. Cuzzocrea, "Retrieving Accurate Estimates to OLAP Queries over Uncertain and Imprecise Multidimensional Data Streams". In: *23rd International Conference on Scientific and Statistical Database Management*, pp. 575-576, 2011
- [18] S. Acharya, V. Poosala, S. Ramaswamy, "Selectivity Estimation in Spatial Databases". In: *ACM International Conference on Management of Data*, pp. 13-24, 1999
- [19] N. Bruno, S. Chaudhuri, L. Gravano, "STHoles: A Multidimensional Workload-Aware Histogram". In: *ACM International Conference on Management of Data*, pp. 211-222, 2001
- [20] D. Gunopulos, G. Kollios, V.J. Tsotras, C. Domeniconi, "Selectivity Estimators for Multidimensional Range Queries over Real Attributes". *The VLDB Journal* 14(2), pp. 137-154, 2005
- [21] University of California-Irvine, *Echocardiogram Dataset*. Available at <https://archive.ics.uci.edu/dataset/38/echocardiogram>
- [22] D. Campos, J. Bernardes, *Cardiotocography Dataset*. Available at <https://archive.ics.uci.edu/dataset/193/cardiotocography>
- [23] F. Yang, D. Milosevic, J. Cao, "Optimising Column Family for OLAP Queries in HBase". *International Journal of Big Data Intelligence* 4(1), pp. 23-35, 2017
- [24] K. Dehdouh, O. Boussaid, F. Bentayeb, "Big Data Warehouse: Building Columnar NoSQL OLAP Cubes". *International Journal of Decision Support System Technology* 12(1), pp. 1-24, 2020
- [25] R. Tardío, A. Maté, J. Trujillo, "Beyond TPC-DS, A Benchmark for Big Data OLAP Systems (BDOLAP-Bench)". *Future Generation Computer Systems* 132, pp. 136-151, 2022
- [26] A. Cuzzocrea, "CAMS: OLAPing Multidimensional Data Streams Efficiently". In: *11th International Conference on Data Warehousing and Knowledge Discovery*, pp. 48-62, 2009
- [27] M. Ceci, A. Cuzzocrea, D. Malerba, "Effectively and Efficiently Supporting Roll-Up and Drill-Down OLAP Operations Over Continuous Dimensions via Hierarchical Clustering". *Journal of Intelligent Information Systems* 44(3), pp. 309-333, 2015
- [28] A. Cuzzocrea, D. Saccà, P. Serafino, "Semantics-Aware Advanced OLAP Visualization of Multidimensional Data Cubes". *International Journal of Data Warehousing and Mining* 3(4), pp. 1-30, 2007
- [29] C.K. Leung, P. Braun, A. Cuzzocrea, "AI-Based Sensor Information Fusion for Supporting Deep Supervised Learning". *Sensors* 19(6), art. 1345, 2019
- [30] Z. Xu, W. Luk, S. Petchulat, "Client-Centric OLAP on Mobile Devices". In: *29th IEEE International Conference on Data Engineering*, pp. 104-111, 2013
- [31] K. Boulil, F. Le Ber, S. Bimonte, C. Grac, F. Cernesson, "Multidimensional Modeling and Analysis of Large and Complex Watercourse Data: An OLAP-Based Solution". *Ecological Informatics* 24, pp. 90-106, 2014
- [32] M. Kim, "Graph OLAP-Based Influence Analytics for Complex Gene and Disease Influence Network". In: *International Conference on Computational Science and Computational Intelligence*, pp. 1749-1753, 2021